

SmartCam

Mehul Goel, Ashley Garcia, Youjia Tong, Chenghao Xiang
Electrical & Computer Engineering, Rice University, Houston, TX 77005 USA

Abstract—Autonomous vehicle navigation in large-scale urban and indoor environments faces critical challenges from GPS unavailability, cumulative localization errors, and sensor failures that could potentially compromise safety. Current solutions rely heavily on GPS modules or centralized mapping systems that struggle with scalability and robustness. We propose SmartCam, a collaborative SLAM framework in which multiple cars share local submaps at common landmarks to build globally consistent maps. A central server coordinates vehicle status updates and collision avoidance by dynamically assigning tasks and unexplored areas. The system itself integrates a sensor-based emergency braking layer for fault tolerance. Through ROS2-based communication and machine-learning-enhanced place recognition, SmartCam enables seamless navigation and mapping in challenging environments [1]. Initial evaluations indicate a more consistent map construction, a lower accumulated localization error, and improved coverage efficiency compared to single-robot SLAM approaches.

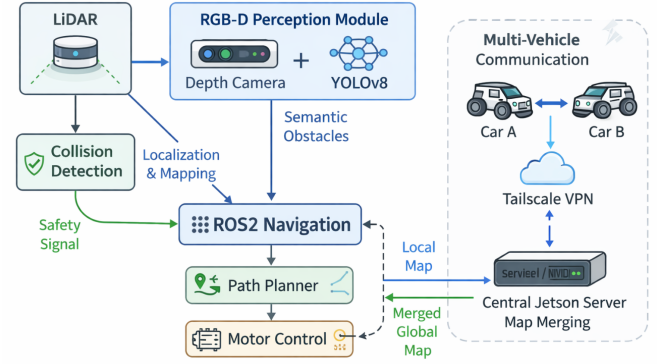
Index Terms—Autonomous driving, Computer vision, Path planning, Embedded AI, Smart mobility, Edge computing.

I. INTRODUCTION

The paradigm shift in autonomous mobile robotics from monolithic single-agent systems to decentralized Multi-Agent Systems (MAS) has unlocked unprecedented capabilities in collective environmental perception and operational redundancy. However, realizing robust Simultaneous Localization and Mapping (SLAM) within such frameworks poses significant computational challenges, particularly when deployed on resource-constrained edge devices [2]. The primary bottleneck lies in the high-fidelity processing of asynchronous sensor data—spanning LiDAR point clouds, IMU inertial frames, and visual streams—that must be synthesized in real time to ensure trajectory coherence.

Our research, titled “Smart Cam,” addresses these challenges through a heterogeneous computing architecture designed to optimize power-to-performance efficiency. By bifurcating the computational workload, we leverage the NVIDIA Jetson Nano for high-level, GPU-accelerated perception—specifically YOLOv8 for real-time object detection—while delegating low-latency sensor fusion and PID-based control loops to the Raspberry Pi 5. This hierarchical design, as seen in Figure 1, enables concurrent execution of deep learning models and high-frequency SLAM optimizations on the HiWonder mobile platform without compromising system stability.

A salient contribution of this project is the implementation of a probabilistic state-space model to mitigate the stochastic noise and temporal latency inherent in heterogeneous sensor fusion. We frame the vehicle’s localization as a probabilistic



SmartCamera System Pipeline

Fig. 1: Overall System Pipeline

state-space model, where the objective is to estimate the Maximum Likelihood Trajectory (MLT) [3]. If we define the state at time t as s_t , our goal is to maximize the joint probability:

$$P(s_{1:T}|o_{1:T}) = \max \prod P(o_t|s_t)P(s_t|s_{t-1})$$

This methodology effectively filters out outliers caused by communication jitter and rolling shutter distortions, providing a mathematically grounded solution to sensor synchronization [4]. This report outlines the preliminary integration of the SLAM Toolbox, the debugging of inter-processor communication protocols, and the initial validation of the factor graph optimization framework [5].

II. METHODOLOGY

A. General Functions

i. SLAM Toolbox and GTSAM: For the multi-agent deployment of the SmartCam fleet across the Rice University campus, a centralized 2D map-merging architecture was selected to balance edge-compute constraints with robust navigation capabilities. To overcome the strict NAT and multicast restrictions in the university’s wireless infrastructure, the communication pipeline uses a Tailscale virtual private network and the Zenoh protocol. This architecture ensures secure, low-latency, unicast data transfer between the peripheral rover units (equipped with Raspberry Pi 4s) and the central mapping server (hosted on a Jetson Orin Nano). Rather than saturating network bandwidth with dense 3D point clouds, the edge units perform local simultaneous localization and mapping (SLAM) to generate highly compressed 2D occupancy grids and serialized pose graphs. These localized map fragments and odometry constraints are continuously

transmitted to the central server. Drawing on graph-based SLAM backend optimization principles, the central server analyzes the incoming data to identify geometric overlaps and inter-robot loop closures. Upon detecting a confident match between two distinct local maps, the server establishes virtual constraints between their respective nodes. A nonlinear least-squares optimizer is then applied to the newly combined pose graph, elastically deforming and aligning the independent trajectories into a single, unified global coordinate frame. This globally consistent 2D occupancy grid is subsequently broadcast to the individual SmartCam units, providing the necessary environmental context for autonomous, decentralized path planning across the campus.

ii. Sensor Fusion Pipeline and Odometry Calibration:

To mitigate the inherent challenges of odometric drift, the data pipeline feeds raw telemetry from HiWonder's LiDAR, depth camera, IMU, and wheel encoders to Toolbox's internal GTSAM-based pose graph optimizer. A primary focus of this implementation is the development of a Kalman filter to fuse the data from the wheel encoder with the IMU measurements. This specifically allows tuning of the covariance matrices, thereby ensuring that the IMU compensates for encoder noise and reduces rotation drift. This modular approach specifically allows for the sequential integration of sensors, starting with a LiDAR baseline and incrementally adding encoders, IMU, and depth landmarks to better quantify the marginal accuracy gains of each modality. The validation of the sensor fusion layers will be conducted through a comparative analysis of four distinct configurations: LiDAR-only, LiDAR+Encoders, LiDAR+Encoders+IMU, and a full stack integration including depth camera landmarks. The end goal will therefore be to quantify these results by comparing the robot's estimated trajectory against a physical ground-truth path using the Root Mean Square Error (RMSE).

iii. YOLOv8n (You Only Look Once, version 8, Nano):

To achieve semantic obstacle avoidance in real-time within the computational constraints of the embedded hardware architecture, as shown in Figure 2, the YOLOv8n (Nano) convolutional neural network was integrated into the primary vision pipeline. The model was aggressively optimized for FP16 precision in TensorRT to maximize inference throughput on the edge GPU, ensuring low-latency detection of dynamic obstacles, such as pedestrians and campus infrastructure. The network processes continuous RGB video streams, regressing bounding box coordinates, and extracting object classification probabilities. To transition from 2D pixel space to actionable 3D navigational space, the spatial centroids and boundaries of the detected bounding boxes are correlated with the time-synchronized depth map, enabling the system to extract the median spatial distance to the target. Upon localized detection, the semantic vision node transforms these coordinates from the camera's frame to the rover's base link and dynamically injects the data into the ROS 2 local costmap. By representing classified objects as lethal inflation zones within the costmap, the primary path-planning algorithm is forced to autonomously recalculate its local trajectory, generating smooth, evasive velocity commands to navigate around the identified hazards while maintaining progress toward the global objective [6].

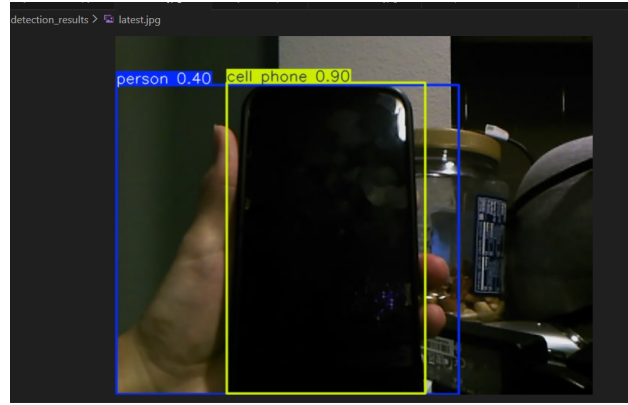


Fig. 2: Object Detection Results: YOLOv8 Depth Camera.

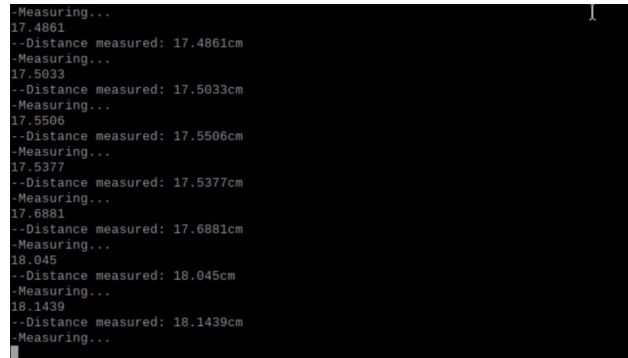


Fig. 3: Preliminary Ultrasonic Measurements.

iv. Automatic Emergency Braking: To ensure the physical safety of the autonomous platform and its surrounding environment, an Automatic Emergency Braking (AEB) subsystem was implemented using an HC-SR04 ultrasonic sensor serving as a deterministic, low-latency failsafe independent of the primary visual and LiDAR-based navigation stacks. The methodology relies on acoustic Time-of-Flight (ToF) principles, where a 40 kHz ultrasonic burst is emitted, and the precise duration of the returning echo is measured to calculate the distance to forward obstacles within an operational range of 2cm to 400cm. To mitigate false-positive actuations caused by multipath acoustic reflections or environmental noise, the raw sensor telemetry is conditioned through a real-time median filter before evaluation. To ensure metric accuracy, the system undergoes a calibration phase in which the user inputs a known distance, and the sensor samples five discrete measurements. These samples characterize the sensor's variance and establish a scaling factor to map ToF to precise distance in centimeters, as demonstrated by the preliminary outputs in Figure 3. The collision detection logic utilizes dynamic Time-to-Collision (TTC) calculations, factoring in the spatial derivative of the filtered distance to determine the relative approach velocity. Upon breaching a predefined critical TTC threshold, the AEB subsystem triggers a high-priority override command. Within the local control architecture, this emergency interrupt preempts all standard localized path-planning velocities, aggressively overriding the motor control signals to force an immediate deceleration and halt the rover, thereby preventing imminent forward collisions.

B. The Overall Execution Flow

i. Exploration: Car A and Car B will be deployed on different corners of a room. Both start running SLAM Toolbox locally, building their own independent maps.

ii. Semantic Navigation: Car A detects a bicycle using YOLOv8n. It projects the bicycle into its local costmap. The Nav2 planner routes the car around the bicycle. The LiDAR points that hit the bicycle are filtered out to prevent them from corrupting the SLAM graph.

iii. Emergency Intervention: A student suddenly steps blindly in front of Car A. The YOLO model is momentarily too slow to catch the frame, but the ultrasonic HC-SR04 sensor detects a Time-to-Collision of 0.5 seconds. The AEB interrupt fires, slamming the brakes. Car A halts safely.

iv. Data Transmission: Throughout this drive, both cars are streaming their compressed pose-graphs and local map data over the Tailscale VPN to the central Jetson server.

v. Global Optimization: Car A and Car B eventually turn into the same hallway. The central server observes that the LiDAR geometry from Car A matches the geometry from Car B. It creates an inter-robot factor constraint in GTSAM.

vi. The Result: The GTSAM optimizer runs, instantly correcting any odometry drift and merging the two separate grids into one unified campus map. The central server broadcasts this global map back to the fleet, allowing Car A to plot a path into the territory that Car B just finished exploring.

III. RESULTS

The current phase of the SmartCam project has successfully transitioned from the baseline leader-follower architecture into the formalized design and component validation of a centralized, multi-agent autonomous system. The primary results of this period include finalized architectural blueprints, validated communication topologies, and integrated algorithmic pipelines engineered for deployment across the Rice University campus.

A. Network Communication

A critical bottleneck identified in early iterations was the inability of standard ROS 2 Data Distribution Service (DDS) to operate on university's restricted, multicast-blocking wireless network. As a result, a novel communication architecture was designed and validated. The system now uses a Tailscale Virtual Private Network (VPN) overlay in combination with the Zenoh protocol. This configuration successfully bridges ROS 2 nodes using highly compressed, unicast point-to-point connections.

B. Failsafe and Semantic Vision Integration

An independent, deterministic safety layer, Automatic Emergency Braking, will be designed using HC-SR04 ultrasonic sensors. For the primary vision pipeline, a YOLOv8n (Nano) convolutional neural network is selected, optimized with TensorRT FP16 precision to maximize inference throughput on the edge GPU. Object coordinates are dynamically injected into the ROS 2 local costmap as lethal inflation

zones, allowing the Nav2 planner to autonomously calculate evasive trajectories while filtering dynamic objects out of the permanent SLAM map.

C. Centralized map merging architecture

To achieve multi-agent environmental awareness without exceeding the computational limits of the embedded hardware, a 2D pose-graph map-merging strategy is formalized. Rather than attempting to transmit bandwidth-heavy 3D point clouds, the edge rovers utilize local SLAM algorithms to generate highly compressed 2D occupancy grids and serialized odometry graphs. The computational load of map merging will be offloaded to a central Jetson Orin Nano hub. This central server is tasked with applying graph-based backend optimization to detect geometric overlaps, establish inter-robot constraints, and elastically deform the independent local graphs into a single, unified global coordinate frame for campus-wide path planning.

D. SLAM and Sensor Fusion Baseline

Initial testing has thus far focused on establishing a stable data pipeline and characterizing raw sensor noise. While full-scale mapping is currently in progress, the LiDAR has been successfully verified and integrated. The next step is to incrementally fuse wheel encoder and IMU data into the SLAM Toolbox backend. This will involve tuning the Kalman Filter to establish a reliable odometric baseline. Following this, the depth camera will be integrated to provide visual landmark constraints. The final validation of this phase will produce a comparative performance table quantifying RMSE for each sensor configuration.

IV. DISCUSSION

With the theoretical framework, network topology, and sub-system methodologies defined, the next phase is the physical deployment of these pipelines on the SmartCam hardware. Upcoming sprints will focus on flashing the optimized TensorRT models to the Jetson Orin Nano, configuring the Zenoh bridges across the Tailscale nodes, and conducting empirical field tests of the AEB system's latency.

Accomplishments and capability. Across simulation and hardware, the project delivered a functional end-to-end autonomy pipeline:

- (i) LiDAR-based 2D SLAM of the leader car that maintains a consistent occupancy-grid structure when closed-loop speed and yaw feedback is enabled.
- (ii) Successfully implemented a visual servoing system of a follower car capable of autonomous ArUco code tracking by converting computer vision data using a depth camera. These results establish a robust baseline for low-cost compute and hobby-grade actuation, as well as the system's current capability level.

CREDIT AUTHOR STATEMENT

All authors contributed equally as collaborators and shared responsibility throughout the project. Each author played a primary

role while also supporting other aspects of the work as part of a horizontal and cooperative team effort.

- **Ashley Garcia:** Conceptualization, SLAM Pipeline & GTSAM Optimization, Methodology, Sensor Fusion, Visualization, Validation, Writing : Original Draft, Cross-Team Coordination
- **Mehul Goel:** Software Development, Automatic Emergency Braking, Hardware Integration, Sensor Calibration, Testing, Writing : Review & Editing
- **Chenghao Xiang:** Conceptualization, YOLOv8 Implementation, Perception Pipeline, Hardware Integration, Methodology, Visualization, Cross-Team Coordination
- **Youjia Tong:** Software Development, Wireless Communication Validation, Experimental Investigation, Validation, Obstacle Detection Integration, Testing, Writing : Review & Editing

All authors participated in discussions, reviewed the manuscript, and approved the final version for publication.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to Professor Jose Moreto and Professor Joseph Young for their guidance, support, and coordination throughout the capstone project. Their advice and encouragement greatly contributed to the development of this work.

REFERENCES

- [1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, and A. Y. Ng, "Ros: An open-source robot operating system," *IEEE International Conference on Robotics and Automation Workshop on Open Source Software*, 2009.
- [2] C. Cadena *et al.*, "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016.
- [3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [4] S. Macenski *et al.*, "Slam toolbox: Unified 2d slam for the ros 2 ecosystem," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.
- [5] F. Dellaert and M. Kaess, "Factor graphs for robot perception," *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017.
- [6] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics and Automation Magazine*, 1997.